

Extending CoFinDiff:  
Incorporating Macroeconomic Conditioning in Financial  
Time Series Generation

Author: Andrew Liu

Faculty Advisor: Professor Weining Shen

University of California, Irvine

Graduating Quarter: Spring 2026

## Abstract

Financial data is limited and finite, yet modern financial modeling increasingly requires large amounts of high-quality data for stress testing, algorithm validation, and scenario analysis. This need is especially amplified by recent advancements in deep learning, where model performance often depends on access to large training datasets. CoFinDiff addresses this challenge by using a diffusion-based approach to generate synthetic financial time series while allowing controllable generation through micro-level conditionings such as trend and realized volatility. This controllability enables synthetic data to be generated with targeted behaviors and statistical properties rather than relying purely on stochastic variation. This project extends the CoFinDiff framework by incorporating macroeconomic conditioning, allowing the model to generate financial time series that respond to market-level variables such as interest rates and the volatility index. By adding macroeconomic context, this work aims to improve the flexibility, interpretability, and regime-awareness of controllable synthetic financial data generation.

## 1 Introduction

Synthetic financial data has become an important research direction because real financial data is limited, sensitive, and difficult to scale in the same way as traditional text, image, or tabular datasets. Since financial time series arise from real market interactions, they are constrained by historical events, market structure, and the limited observation of rare regimes. As a result, synthetic data generation has emerged as a useful tool for producing realistic datasets that can support privacy-preserving data sharing, model development, validation, and analysis [1]. In this context, synthetic data is not only a way to increase sample size, but also a way to create controlled and reusable data for financial modeling tasks. However, generating useful synthetic financial time series remains challenging. This difficulty is especially relevant as deep learning methods become more common in finance, where broad coverage of market behaviors is needed for models to generalize beyond observed historical conditions while maintaining robustness [2]. Increasing sample size alone is not sufficient because financial time series exhibit well-documented stylized facts, including fat-tailed returns, extreme fluctuations, and volatility

clustering [3]. Therefore, useful synthetic financial data should not only expand the quantity of available data, but also reproduce the empirical behaviors that make financial markets difficult to model.

Traditional financial data generation methods, such as geometric Brownian motion, ARCH, and GARCH-based models, provide interpretable statistical structure but often struggle to reproduce the full complexity of real market behavior [4]. Recent work has therefore explored deep generative methods, including generative adversarial networks, variational autoencoders, and diffusion models (to be discussed more in-depth in section 2.1). While these machine learning approaches have expanded the potential realism of synthetic financial time series, CoFinDiff introduced a new layer of controllability. Rather than generating return windows without explicit statistical targets, CoFinDiff conditions generation on trend and realized volatility, allowing specified micro-level statistical properties to be reflected in the generated output. CoFinDiff shows promising results in reliable conditional generation, producing financial time series that better satisfy specified trend and volatility conditions than GAN-based baselines, while also improving adherence to stylized facts and generation diversity [5].

Although this added controllability makes CoFinDiff a strong foundation for controllable financial time-series generation, its conditioning is limited to variables calculated from the generated return series itself. Trend and realized volatility describe the internal behavior of a return window, but they do not directly represent broader market regimes. Macro-level regimes can affect financial time series because periods of higher interest rates, inflation uncertainty, or broader economic instability are often associated with changes in the volatility of stock returns [4]. This leaves an important gap because market behavior is also shaped by macroeconomic and market-level variables such as interest rates and volatility-index conditions. Extending controllability to macroeconomic variables can therefore make synthetic financial generation more context-aware and better suited for regime-based market analysis.

This project builds on the foundational architecture of CoFinDiff and expands its controllable parameters to include macroeconomic variables. Specifically, the model augments the original micro-level controls of trend and realized volatility with market-level variables such as interest rates and the volatility index, using cross-attention and Feature-wise Linear Modulation to inject macroeconomic information

into the generation process (to be discussed more in-depth in section 2.3 and 3.3). The extended framework is evaluated along three dimensions: statistical fidelity, diversity, and conditional adherence. Statistical fidelity is assessed through stylized facts such as fat tails and volatility clustering; diversity is measured using distance-based comparisons among generated samples; and conditional adherence is evaluated by comparing generated return windows against specified trend and realized-volatility targets while examining behavior across macroeconomic regimes. Through these experiments, this work investigates whether adding macroeconomic context improves the flexibility, interpretability, and regime-awareness of controllable synthetic financial time-series generation.

## 2 Background

This section provides the technical background needed to understand the proposed extension of CoFinDiff. It first reviews and evaluates existing approaches to synthetic financial time-series generation. It then discusses denoising diffusion probabilistic models (DDPMs) in greater depth, since they serve as the base architecture used by CoFinDiff. Next, it introduces the two conditioning mechanisms explored in this project: cross-attention and feature-wise linear modulation (FiLM). Finally, it provides an overview of the original CoFinDiff framework. Together, these topics establish why diffusion-based controllable generation is an appropriate foundation for extending synthetic financial time-series generation with macroeconomic conditioning.

### 2.1 Financial Time Series Generation

Synthetic financial time-series generation is achieved by having a model learn the distributional and temporal structure of real market data so that new realistic sequences can be sampled. Early approaches relied on statistical models such as geometric Brownian motion, ARCH, and GARCH. These models provide interpretable assumptions about drift, volatility, and conditional variance, but they are limited in their ability to represent the non-linear and high-dimensional structure of financial markets [6], [7]. To better capture the complexity of financial time series, researchers have increasingly turned to deep generative models for financial and general time-series generation.

Generative adversarial networks (GANs) model data generation as a minimax game between two players: a generator and a discriminator. The generator learns to create synthetic samples, while the discriminator learns to distinguish generated samples from real ones [8]. This adversarial setup allows the generator to improve by learning from the discriminator's feedback. For time-series data, TimeGAN extends this idea by combining adversarial learning with supervised temporal learning in a learned embedding space, allowing the model to better preserve sequential dynamics [9]. This temporal awareness makes GAN-based models attractive for financial data generation. However, GANs can be difficult to train because adversarial optimization is often unstable and may suffer from mode collapse, where the generator produces samples with limited diversity [10]. In financial time-series generation, this is a meaningful limitation because synthetic data should capture a range of market behaviors rather than repeatedly generate similar patterns.

Variational autoencoders (VAEs) take a different approach by learning a probabilistic latent representation of the data. In a VAE, an encoder maps input data into a latent distribution, and a decoder reconstructs data from samples of that latent space. Training consists of maximizing the evidence lower bound (ELBO), which balances reconstruction quality with regularization of the latent representation [11]. TimeVAE adapts this framework for multivariate time-series generation by incorporating temporal structure and interpretable components such as trend and seasonality [12]. Compared with GANs, VAEs are generally more stable to train because they optimize a reconstruction-based probabilistic objective rather than an adversarial game. However, they may generate overly smooth samples because reconstruction losses can encourage averaging over uncertainty, which can weaken extreme behaviors in the generated series.

Recently, diffusion models have become one of the leading generative approaches because of their stable training objectives, strong sample quality, and flexible conditioning capabilities. At a high level, diffusion models incrementally add noise to the input and train a neural network to reverse this corruption process. During sampling, instead of mapping random noise to data in a single pass, diffusion models iteratively transform noise into structured samples [13]. This progressive denoising process allows

the model to learn complex data distributions through many smaller reconstruction steps. Recent time-series work such as Diffusion-TS shows that diffusion-based models can generate high-quality multivariate time series and can be naturally extended to both unconditional and conditional generation tasks [14]. In the financial domain, CoFinDiff builds on this advantage by using diffusion not only for realistic generation, but also for controllable generation based on specified financial properties [5]. For this project, diffusion is the most appropriate foundation because it offers a stable training objective, strong high-dimensional representation learning, and a compatible framework for incorporating conditioning mechanisms on both micro-level and macro-level variables.

## 2.2 Denoising Diffusion Probabilistic Models

Denoising diffusion probabilistic models (DDPMs) define generation as the reverse of a gradual corruption process. In the forward process, Gaussian noise is progressively added to a clean data sample  $x_0$  over  $T$  timesteps, until the sample approaches pure noise. This can be defined by a Markov chain:

$$q(x_t|x_{t-1}) \sim N(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I), \quad (1)$$

where  $\beta_t$  is a variance schedule that controls the amount of noise added at timestep  $t$  [13]. A noisy sample at any timestep can be directly sampled from the original data by using the reparameterized form where  $\alpha_t = 1 - \beta_t$  and  $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ :

$$q(x_t|x_0) \sim N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I). \quad (2)$$

This formulation is important because it allows the model to train on arbitrary timesteps without sequentially applying every noising step, saving significant time and computational resources.

In the reverse process, the model learns to denoise  $x_t$  back toward  $x_0$ . DDPMs commonly train a neural network to predict the noise that was added to the original data. The simplified training objective follows a fairly standard quadratic loss function:

$$L = \left( E_{t, x_0, \epsilon} \left[ \left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2 \right] \right), \quad (3)$$

where  $\epsilon \sim N(0, I)$  is the true noise and  $\epsilon_\theta(x_t, t)$  is the predicted noise [13]. This objective reduces training to a stable supervised problem, making DDPMs more attractive for reliable generative modeling.

The neural network used for denoising is often a U-Net. Originally introduced for biomedical image segmentation, U-Net uses a downsampling path to capture multi-scale context and an upsampling path to reconstruct detailed outputs, with skip connections linking corresponding resolutions [15]. In diffusion models, this structure is useful because the denoising network must capture both global structures as well as local details. For financial time-series generation, this idea can be adapted by representing time-series windows in a two-dimensional format suitable for convolutional denoising, as done in CoFinDiff through a wavelet-based transformation [5].

### 2.3 Conditional Generation

Conditional generation extends a generative model from learning  $p(x)$  to learning  $p(x|c)$ , where  $c$  represents additional information that should guide the generated output. In financial time-series generation, conditioning is important because users often want synthetic samples that reflect specific behaviors or regimes rather than only random samples from the learned data distribution. Conditioning information can be injected into a neural network in several ways, including direct concatenation, additive embeddings, attention mechanisms, and feature-wise modulation. Simpler approaches such as concatenation or additive embeddings are easy to implement, but they may provide less expressive control because they do not directly determine how intermediate features should be selected, scaled, or shifted throughout the network. For these reasons, this project focuses on cross-attention and FiLM, which provide stronger mechanisms for injecting conditioning information into intermediate model representations.

Cross-attention is a conditioning mechanism that allows one representation to dynamically attend to another. It builds on the attention mechanism introduced in the Transformer architecture, where queries, keys, and values are used to compute weighted combinations of information. The scaled dot-product attention operation is:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (4)$$

where  $Q$  represents queries,  $K$  represents keys,  $V$  represents values, and  $d_k$  is the key dimension [16]. In self-attention, the queries, keys, and values come from the same sequence. In cross-attention, the model's internal hidden states provide the queries, while the conditioning information provides the keys and values. The attention scores measure how strongly each query matches each condition key, and these scores are used to weight the corresponding values which are injected into the hidden representation accordingly. In the context of DDPM-style generation, cross-attention is particularly suitable because scalar conditions can be embedded as condition tokens and injected into the denoising network at each reverse diffusion step.

Feature-wise Linear Modulation (FiLM) provides a different conditioning strategy. Instead of using attention weights, FiLM learns scale and shift parameters that modulates intermediate neural network values. Given an intermediate feature representation  $h$  and a conditioning vector  $c$ , FiLM computes:

$$FiLM(h|c) = \gamma(c) \odot h + \beta(c), \quad (5)$$

where  $\gamma(c)$  and  $\beta(c)$  are learned functions of the conditioning input, and  $\odot$  denotes element-wise multiplication [17]. FiLM is attractive because it provides a direct and efficient way to let external conditions influence the model's internal representations. This is especially fitting for financial data generation, because scalar variables such as interest rates and the volatility index can be easily mapped into feature-wise transformations that adjust how the denoising network processes the generated time series.

## 2.4 The CoFinDiff Framework

CoFinDiff, or Controllable Financial Diffusion, is a diffusion framework designed for controllable synthetic financial time-series generation. Its novelty lies in the addition of user-specified financial conditions that guide generation. Specifically, CoFinDiff conditions generate on two micro-level

properties of a return window: trend and realized volatility [5]. As conditions, trend controls the overall direction of the sequence, while realized volatility controls the magnitude of fluctuations in the corresponding window. By conditioning the diffusion model on these values, CoFinDiff allows users to specify the financial behavior that generated samples should exhibit.

The CoFinDiff pipeline first transforms financial data into a diffusion-suitable representation by applying a Haar wavelet transform, converting one-dimensional return sequences into two-dimensional image-like representations. This allows a U-Net-based diffusion model to process financial time series using convolutional denoising [5]. The model is trained using the standard DDPM objective, where noise is added to transformed return windows and the denoising network learns to predict the added noise. Conditioning information is injected into the model so that the reverse diffusion process is guided by specified trend and realized-volatility targets. This project follows the same overall preprocessing pipeline and architectural workflow, with the addition of macro-level conditions.

CoFinDiff evaluates generated data using several criteria: statistical fidelity, conditional accuracy, diversity, and downstream usefulness. The reported results show that CoFinDiff generates financial time series that better satisfy stylized facts such as fat tails and volatility clustering, better follow specified trend and volatility conditions, and maintain greater diversity than GAN-based and VAE-based baselines [5]. These results make CoFinDiff the primary foundation for this project. This project uses a simplified evaluation framework that excludes downstream usefulness in order to limit scope, while adding evaluation across macroeconomic regimes.

### 3 Methods

This section describes the methodology used to extend CoFinDiff with macroeconomic conditioning. The project follows the original CoFinDiff framework as the foundation, but adapts the data source, conditioning variables, and model architecture to support both micro-level and macro-level control. The methodology consists of four main components: data collection, preprocessing, model design, and evaluation. First, daily financial and macroeconomic data are collected from premium and

public APIs. Next, the data is preprocessed into fixed-length return windows with associated micro and macro conditions. Then, two diffusion-based conditioning architectures are implemented for comparison: a cross-attention model and a cross-attention plus FiLM model. Finally, the generated samples are evaluated based on statistical fidelity, conditional adherence, and diversity.

### 3.1 Data Collection

While the original CoFinDiff project uses 1-minute intraday JPX FLEX data from January 1, 2015 to December 31, 2021, this project opts to use daily-frequency data of adjusted closing price accessed through the Alpha Vantage API. Because daily data provides substantially fewer observations than intraday data, this project expands the data collection from the 12 stocks used in CoFinDiff to the top 100 U.S. companies by market capitalization, and uses a fixed 20-year sample period from January 1, 2000 through December 31, 2019. This window provides a long daily-frequency sample with multiple market regimes while keeping the data collection and preprocessing scope consistent across different stocks and macro variables. Since macro-level conditions cannot be directly computed from return windows, daily interest rate data is collected from the Alpha Vantage API, and daily volatility index data is collected from the Yahoo Finance API.

### 3.2 Data Preprocessing

Preprocessing converts raw adjusted daily closing price into fixed-length, two-dimensional training samples compatible with the diffusion model, with corresponding macroeconomic data attached for model training. This preprocessing procedure largely follows the original CoFinDiff workflow, while adapting the pipeline to daily U.S. equities and externally observed macroeconomic variables.

For each ticker, adjusted closing prices  $p_t$  are loaded over the common sample period described in Section 3.1 and converted into daily log returns:

$$r_t = \log(p_{t+1}) - \log(p_t). \quad (8)$$

Each return series is sliced into non-overlapping windows of length  $T = 64$  using strides of length 64, and each return is multiplied by 100 to convert decimal into percentage-point units. All windows across

all tickers are then pooled into a single array to compute the global standard deviation, which is used to divide each window, bringing returns to a consistent  $O(1)$  magnitude while preserving relative volatility differences across windows. The global standard deviation is also stored for the reverse transformation during inference, allowing model outputs to be mapped back to real log-return units.

Each globally-scaled 64-day return window is then transformed from one-dimension array to two-dimension representation via Haar wavelet transformation. The transform repeatedly pairs adjacent values, replacing each pair with their average and their difference, producing a mean series and a difference series at every decomposition level. This halves the series length at each step until a full hierarchy of difference coefficients is created. With padding, these series are stacked into a two-dimensional image  $x_0 \in \mathbb{R}^{T/2 \times M}$ , where  $M = \lfloor \log_2 T \rfloor + 1$ , creating a data format compatible with our U-Net based diffusion model.

Two micro-level conditions are computed from the return windows after the scaling step by 100 but before the global standard-deviation scaling, so they reflect each window-level magnitude rather than a globally normalized one. More specifically, trend is calculated as:

$$\hat{\mu} = \sum_{t=1}^T r_t, \quad (9)$$

and realized volatility is calculated as:

$$\hat{\sigma} = \sum_{t=1}^T r_t^2, \quad (10)$$

where  $r_t$  is log return at time  $t$ . Note that realized volatility is computed as the sum of squared returns, following the original CoFinDiff condition definition, rather than the conventional square-root form.

Two macro-level conditions, interest rate and volatility index, are derived by averaging the corresponding daily external series over the dates spanned by each window. All four conditions are stored in raw form during preprocessing; z-score standardization is applied on the fly during the training step before encoding.

Each preprocessed example thus consists of the wavelet-transformed return image and its four associated raw conditions. This pipeline yields a dataset of fixed-size, model-ready samples that pair a diffusion target with aligned micro- and macro-level conditioning information.

### 3.3 Model

Recall that this project extends CoFinDiff’s diffusion architecture to support four conditioning inputs: the original two micro-level conditions, trend and realized volatility, alongside two new macro-level conditions, interest rate and volatility index. Because the macro-level variables are externally observed rather than derived from the return window itself, the architecture must integrate heterogeneous micro- and macro-level information into the denoising process. This project proposes two architectures: a cross-attention model (CA) and a cross-attention plus FiLM model (CA-FiLM). Both variants embed the four standardized scalar conditions into learned condition representations, but they differ in how micro- and macro-level information are routed through the denoising network.

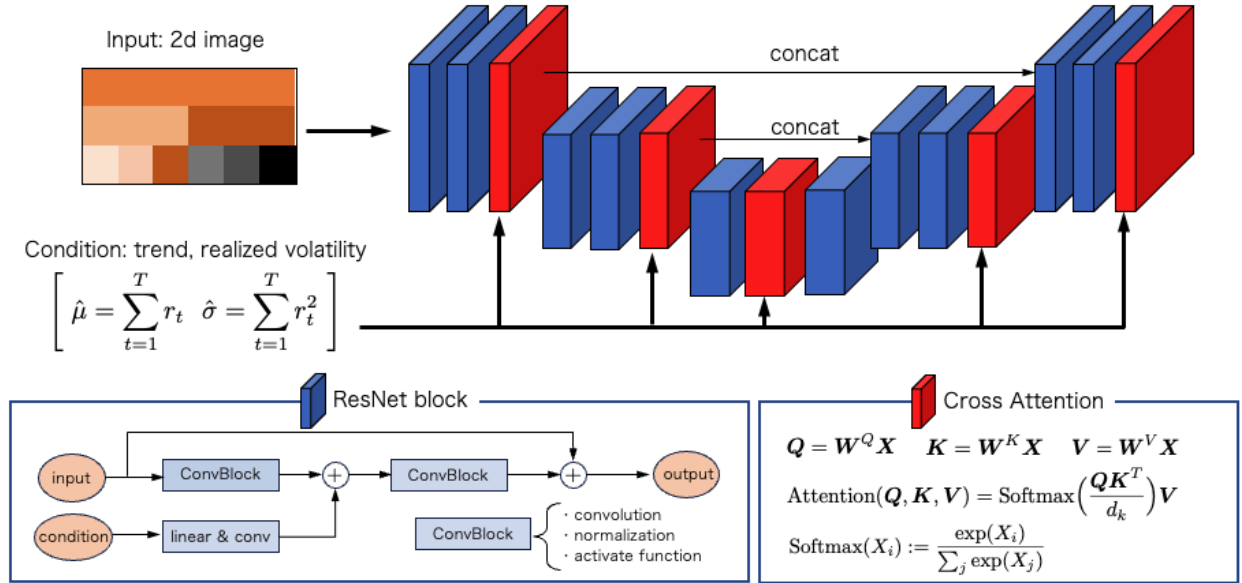
#### 3.3.1 Base Architecture

The base model is a U-Net-based denoising network that takes as input the noisy wavelet-transformed return image  $x_t$ , where the clean target  $x_0 \in \mathbb{R}^{T/2 \times M}$  is produced by the Haar wavelet transform. This image-like representation allows the diffusion model to process financial return windows using convolutional layers, following the CoFinDiff architecture [5]. A U-Net backbone is well suited to this task because it contains a downsampling path that captures multi-scale structure as well as an upsampling path that reconstructs fine-grained detail at full resolution, while skip connections preserve local details that would otherwise be lost during downsampling [13] [15].

The downsampling path is made of encoder blocks that apply a sequence of convolutional layers to extract feature maps at increasingly compressed resolutions, allowing the model to represent both local patterns and overall structure. The upsampling blocks, on the other hand, reconstruct the denoised representation by progressively restoring spatial resolution, using skip connections from corresponding

encoder resolutions to recover detail lost during downsampling. This encoder-decoder U-Net backbone is shared by both proposed variants; the original CoFinDiff architecture is shown in Figure 1.

The diffusion timestep  $t$  is encoded with a linear positional embedding and injected into intermediate U-Net blocks following the standard DDPM setup, which tells the network how much noise is present at the current diffusion step [13]. Timestep conditioning is identical for both variants and is separate from the financial condition representation described above.



**Figure 1. Overview of the CoFinDiff diffusion architecture.**

The wavelet-transformed return image is denoised by a U-Net backbone, while trend and realized volatility conditions are embedded and injected through cross-attention.

### 3.3.2 CA Variant

The cross-attention variant injects the unified condition tensor through the standard cross-attention mechanism [16], where hidden feature maps from selected U-Net resolutions are reshaped into query tokens, while the embedded condition representation is projected into keys and values. This allows each spatial position in the denoising feature map to selectively attend to the most relevant conditioning information. Cross-attention is applied at the bottleneck and decoder stages, where the model can use condition tokens to guide reconstruction during the reverse diffusion.

This design allows the conditioning information to influence the denoising network dynamically. In particular, the model can learn to attend at different levels with regard to trend, realized volatility, interest rate, or VIX, depending on the current timestep and intermediate representation. However, because the conditions in this project are scalar-valued, cross-attention may not always provide the most direct pathway for macroeconomic information to affect feature activations. This motivates the second architecture, which augments cross-attention with FiLM.

### 3.3.3 CA-FiLM Variant

The second variant extends the CA architecture by adding Feature-wise Linear Modulation (FiLM) at each residual block in the U-Net, which effectively performs a linear transformation on the cross-attention values. It uses the same unified condition representation as the CA model, but uses only micro condition embeddings to provide keys and values for cross-attention, and uses macro condition embedding to train learned functions that produce per-channel scale and shift parameters,  $\gamma(c)$  and  $\beta(c)$ . These parameters modulate intermediate feature maps directly according to the FiLM operation introduced in Section 2.3, allowing the macro condition input to shift the magnitude and offset of internal feature maps [17].

This hybrid design adds a persistent conditioning pathway alongside cross-attention. Whereas cross-attention lets the model selectively query condition information at specific spatial locations, FiLM adjusts every channel of the intermediate representation based on the same condition input. Together, the two mechanisms provide both selective and direct conditioning, where cross-attention determines which condition information is relevant, while FiLM globally augments how the network processes these features under different financial contexts. Comparing the CA and CA-FiLM variants therefore tests whether the additional FiLM pathway improves conditional adherence to macroeconomic variables or introduces unnecessary architectural complexity.

### 3.3.4 Training & Hyperparameter Tuning

Both the CA and CA-FiLM models are trained using the AdamW optimizer with a learning rate of  $1.4\text{e-}6$ , weight decay of 0.01, and a linear warmup of 20 steps, for up to 3000 epochs with early

stopping after 100 epochs of no validation improvement. A batch size of 32 is used throughout. To support classifier-free guidance, each training step applies condition dropout with probability  $p_{uncond} = 0.2$ , which zeros out the entire condition representation so the model learns both conditional and unconditional generation. Additionally, with probability 0.5, the condition vectors within a batch are randomly permuted across samples before being passed to the model. This shuffling acts as a regularizer that discourages the model from learning spurious correlations between specific return windows and their associated conditions.

Hyperparameter selection is performed using Weights & Biases sweeps, with the search space covering the diffusion beta schedule (linear, cosine), the base channel width (16, 32, 64), the number of residual blocks per level (2, 3), and the learning rate (1e-4 to 1e-6). Iterative sweeps show that both models achieve optimal performance with the same hyperparameters. At inference, samples are generated using classifier-free guidance with a guidance scale of 7.0, where the final noise prediction is computed as a weighted combination of the conditional and unconditional predictions, with the guidance scale controlling how strongly the generated output is steered toward the specified condition values.

### 3.4 Evaluation

Generated samples are evaluated along three dimensions: 1) statistical fidelity, 2) diversity, and 3) conditional adherence. Statistical fidelity captures two stylized facts of financial returns: fat tails, measured by Fisher's excess kurtosis per window, and volatility clustering, measured by the autocorrelation of squared returns at different time lags [3]. Diversity is quantified as the mean pairwise Euclidean distance as well as the mean pairwise Dynamic Time Warping distance across all generated sample pairs. Conditional adherence is reported as MAE and Pearson correlation between target and achieved condition values, computed separately for micro-level conditions (trend, realized volatility) and macro-level conditions (interest rate, volatility index). The original CoFinDiff experiments also evaluated effectiveness on downstream tasks, but this project omits it to limit scope.

To generate samples for this evaluation framework, the following condition values are used to cover different trend, volatility, and macroeconomic regimes:

- Trend: [-10, -5, 0, 5, 10]
- Realized volatility: [30, 40, 50, 60, 70]
- Interest rate / volatility index: [(5, 20), (8, 35)]

This grid-like condition space consists of five trend values, five realized volatility values, and two macro-level specifications, producing 50 total conditioning combinations. For each combination, both the CA and CA-FiLM models generate 5,000 samples, allowing the evaluation to capture model behavior across a broad range of micro- and macro-level conditions.

## 4 Results

Overall, both CA and CA-FiLM model variants reproduce important characteristics of financial data, including fat-tailed behavior and volatility clustering, while maintaining non-trivial diversity across generated samples. Both models also exhibit clear and successful response to micro-level conditions, but do not respond as well to macro-level conditions.

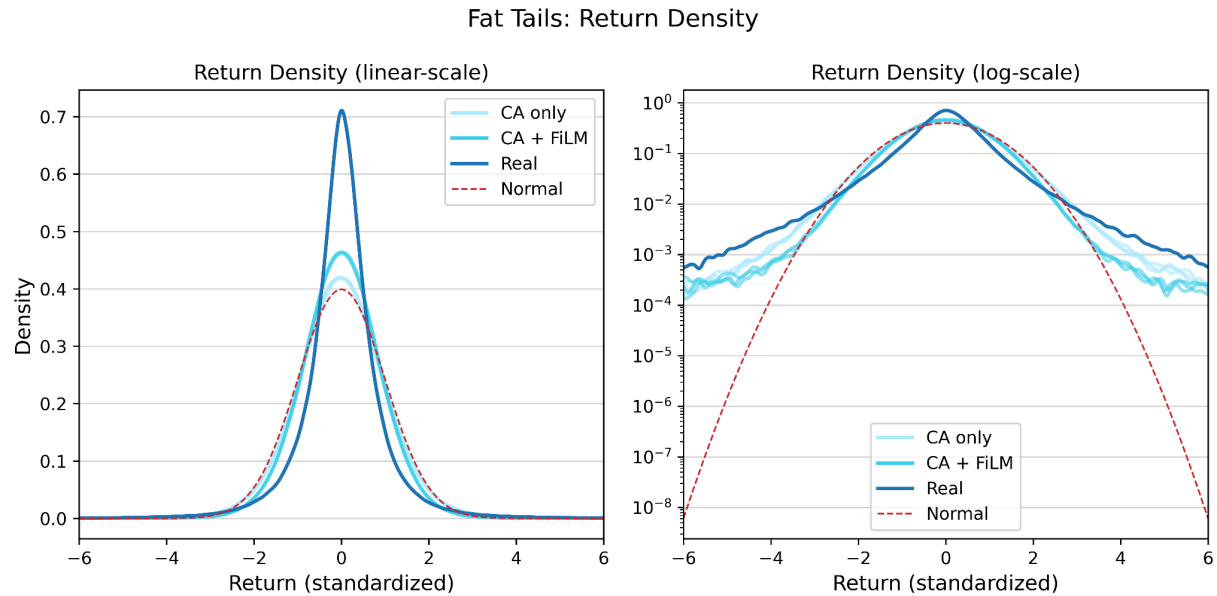
### 4.1 Statistical Fidelity

A central evaluation question is whether the generated samples resemble real financial data and preserve key financial stylized facts. Statistical fidelity evaluates whether these basic empirical properties are retained in the generated return series. This project focuses on two main properties: fat-tailed returns and volatility clustering. These are important because synthetic financial data that appears visually plausible but lacks extreme-return behavior or temporal dependence in volatility would be less useful for real-world applications such as risk modeling, stress testing, and model validation. Therefore, the generated samples are compared against real return windows and a Gaussian reference distribution using return-density plots, excess kurtosis, and autocorrelation of squared returns.

#### 4.1.1 Fat tail returns

Real financial returns often contain more extreme observations than would be expected under a normal distribution, and a useful synthetic generator should preserve this risk-relevant behavior. Figure 2 provides visual confirmation that both the CA and CA-FiLM models learn fat-tailed behavior, as the

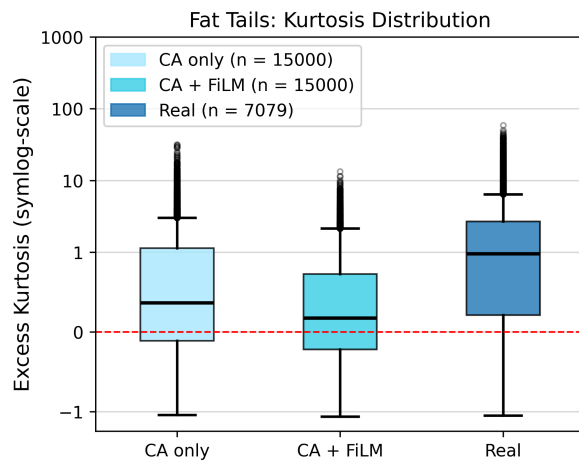
generated return densities have sharper central peaks and heavier tails than the Gaussian baseline.



**Figure 2. Exhibition of fat tail returns relative to a Gaussian baseline..**

Left: KDE of standardized returns on a linear scale. Right: the same densities on a log-scale y-axis, which makes tail behavior more visible. Both real returns and samples from the CA and CA-FiLM models display substantially heavier tails than a fitted normal distribution, confirming that the models reproduce this key stylized fact of financial returns.

This pattern is also supported quantitatively by Fisher's excess kurtosis, which measures how much a distribution exhibits extreme values relative to a normal distribution, where positive values indicate heavier-than-Gaussian tails. Although both generated models produce lower excess kurtosis than the real return data, their positive mean and median kurtosis values show that they still effectively capture the non-normal tail behavior of financial returns.

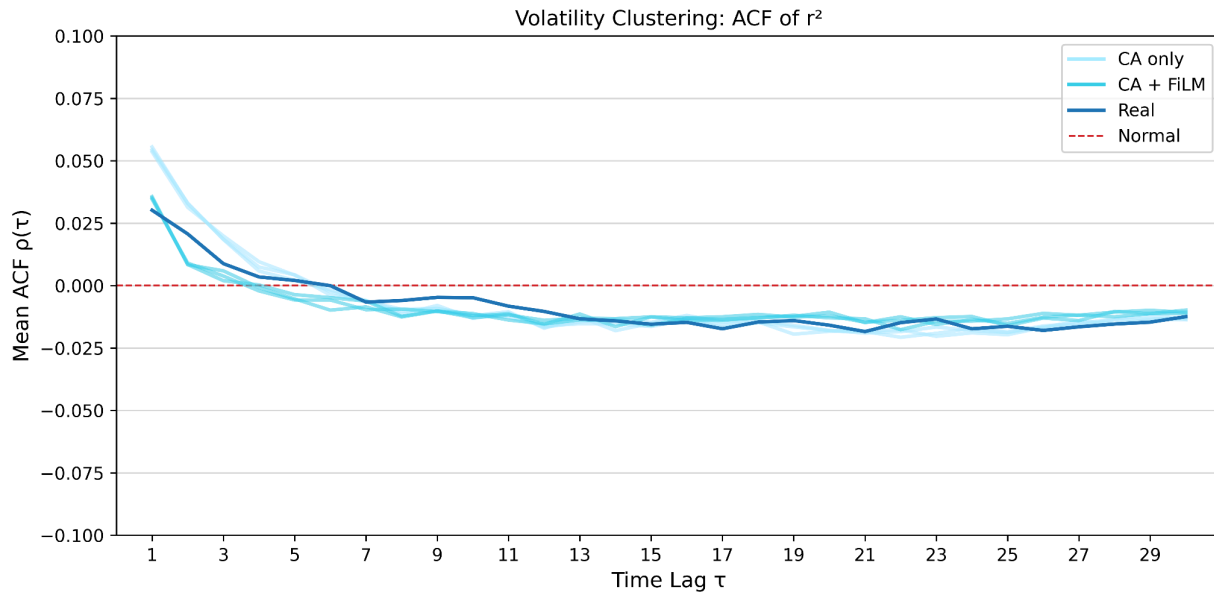


**Figure 3. Distribution of excess kurtosis across generated and real samples.**

Box plot of Fisher's excess kurtosis on a sym-log scale. Generated samples from both models are positively skewed above the Gaussian baseline (marked with red dashed line), consistent with fat-tailed returns, though with lower median and spread than real data.

### 4.1.2 Volatility Clustering

In real financial markets, large return fluctuations tend to be followed by similar large fluctuations, and small fluctuations tend to be followed by similar small fluctuations. This phenomenon is known as volatility clustering. To evaluate this, the autocorrelation function (ACF) of squared returns is examined, since ACF measures how closely a sequence of data points relates to its own past values at different time intervals, also known as lags. Figure 4 shows that generated samples reproduce autocorrelation values higher at short lags and decay over time. The presence of this decaying autocorrelation pattern suggests that the diffusion models capture some temporal dependence in volatility, despite generating fixed-length windows through a wavelet-transformed representation.



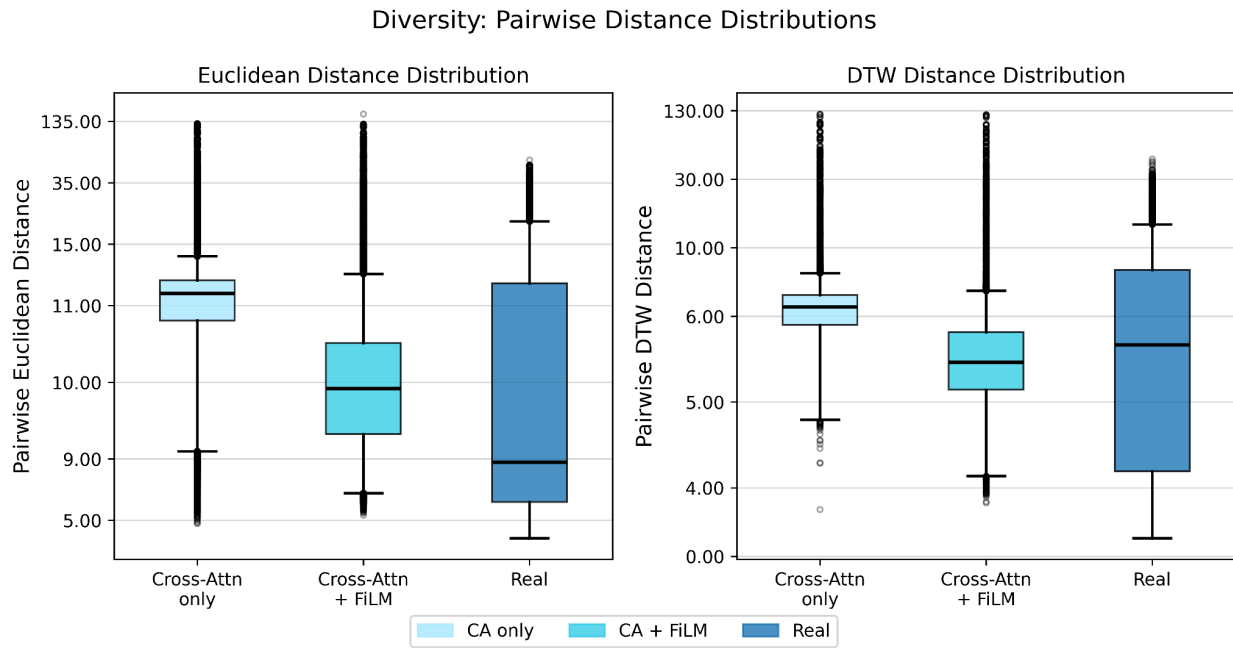
**Figure 4. Exhibition of short-lag autocorrelation consistent with volatility clustering.**

Both models and real data show positive autocorrelation at short lags that decays gradually, in contrast to the near-zero ACF of Gaussian noise, indicating that periods of high and low volatility tend to persist over time.

## 4.2 Diversity

Synthetic financial data must avoid collapsing into repetitive samples to be useful, making diversity another key property to evaluate. Diversity is measured using both Euclidean distance and Dynamic Time Warping (DTW) distance, capturing pointwise differences and sequence-shape differences respectively. Results show that both models maintain meaningful sample diversity, with pairwise distance

distributions comparable to or exceeding that of the real-data reference, suggesting that the proposed conditioning mechanisms retain variation in the generation process. It is worth noting, however, that higher diversity does not strictly indicate better performance, since stronger conditioning may naturally reduce inter-sample variation by anchoring generation more tightly to specified conditions. The comparable diversity observed here is therefore best interpreted as confirming that the models do not suffer from mode collapse, rather than as a direct measure of generation quality.



**Figure 5. Pairwise distance distributions confirm non-trivial sample diversity across generated and real data.**

Box plots of pairwise Euclidean (left) and DTW (right) distances between samples, shown on a sym-log scale. Both models produce inter-sample distances of comparable magnitude to real data, indicating that generated samples are in fact diverse.

### 4.3 Condition Adherence

Finally, conditional adherence is evaluated to determine whether generated samples respond to the input conditions used during sampling. Unlike stylized facts and diversity, which measure general realism, conditional adherence directly tests model controllability. This distinction is central to the project because the goal is not only to generate realistic financial time series, but also to generate them under specified micro-level and macro-level conditions.

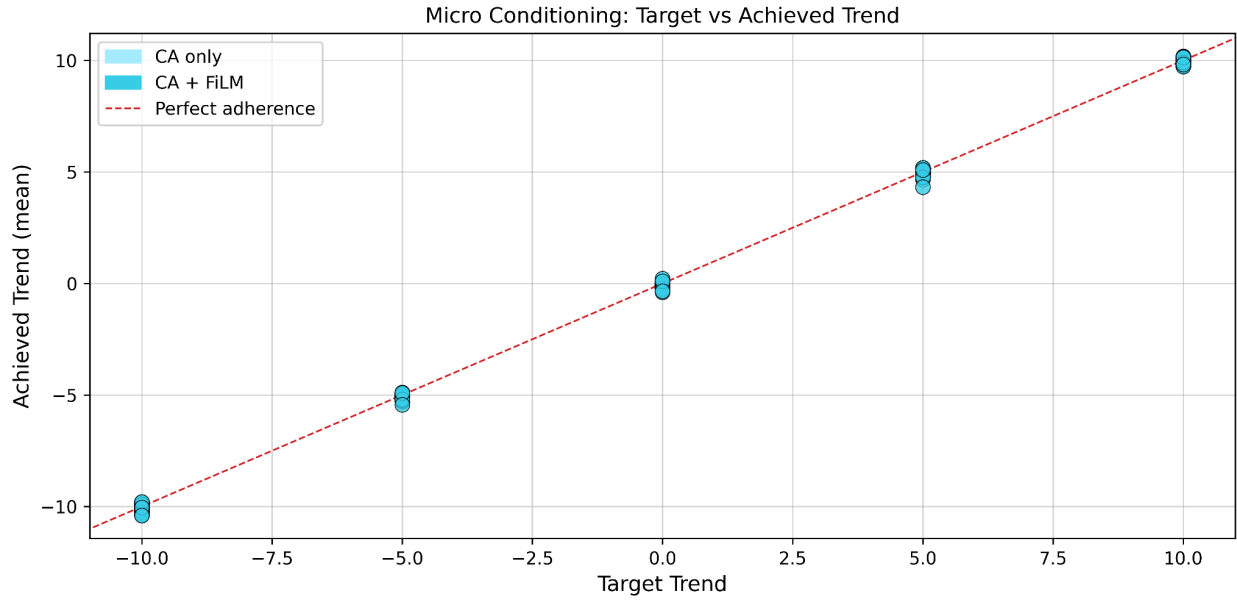
#### 4.3.1 Micro-level Condition Adherence

To evaluate responsiveness to micro-level conditioning, target micro-level conditions are plotted against the achieved conditions computed from generated samples. For trend, both model variants closely follow the ideal reference line, indicating that generated windows successfully match the direction and magnitude specified by the target trend condition. For realized volatility, both models still noticeably track the specified values, though with greater drift. More specifically, the CA model tends to produce samples with higher-than-specified realized volatility, whereas the CA-FiLM model produces samples with realized volatility centered more closely around the target values.

The quantitative results further support these observations. For trend, both models achieve nearly perfect Pearson correlation, with values of 1.0000 for the CA model and 0.9996 for the CA-FiLM model. The CA model also achieves a lower trend MAE of 0.0763, compared to 0.1893 for CA-FiLM, suggesting slightly more precise trend control. For realized volatility, both models maintain strong Pearson correlations, with values of 0.9814 for CA and 0.9660 for CA-FiLM, but the CA-FiLM model achieves a much lower MAE of 3.1092 compared to 12.4581 for CA. Overall, both models exhibit strong micro-level conditioning, with especially strong trend adherence and meaningful realized-volatility control. The greater error for realized volatility is still reasonable because realized volatility is harder to control than trend, since it depends on the magnitude of fluctuations across the full window rather than the net direction alone.

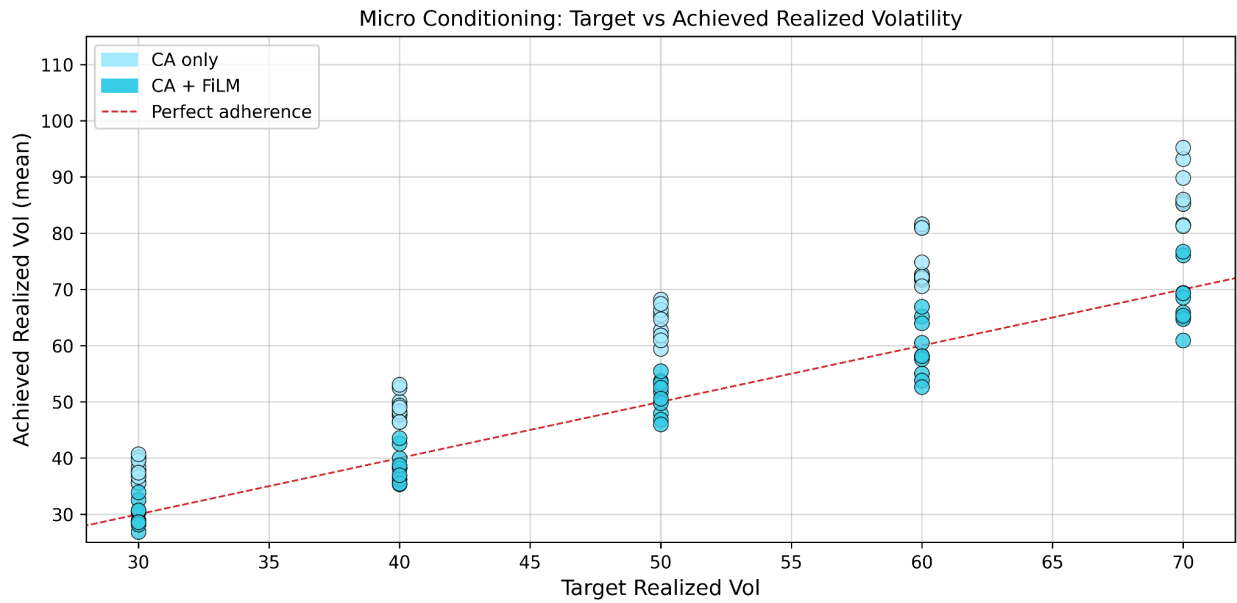
| Micro-condition | Trend    |               | Realized Volatility |               |
|-----------------|----------|---------------|---------------------|---------------|
| Model           | CA Model | CA-FiLM Model | CA Model            | CA-FiLM Model |
| Mean MAE        | 0.0763   | 0.1893        | 12.4581             | 3.1092        |
| Pearson R       | 1.0000   | 0.9996        | 0.9814              | 0.9660        |

**Table 1. Mean MAE and Pearson R metrics for micro-level conditions computed for both models.**



**Figure 6. Both models closely track target trend values.**

Points from both models fall nearly on the perfect-adherence line, indicating precise micro-level conditioning on trend.



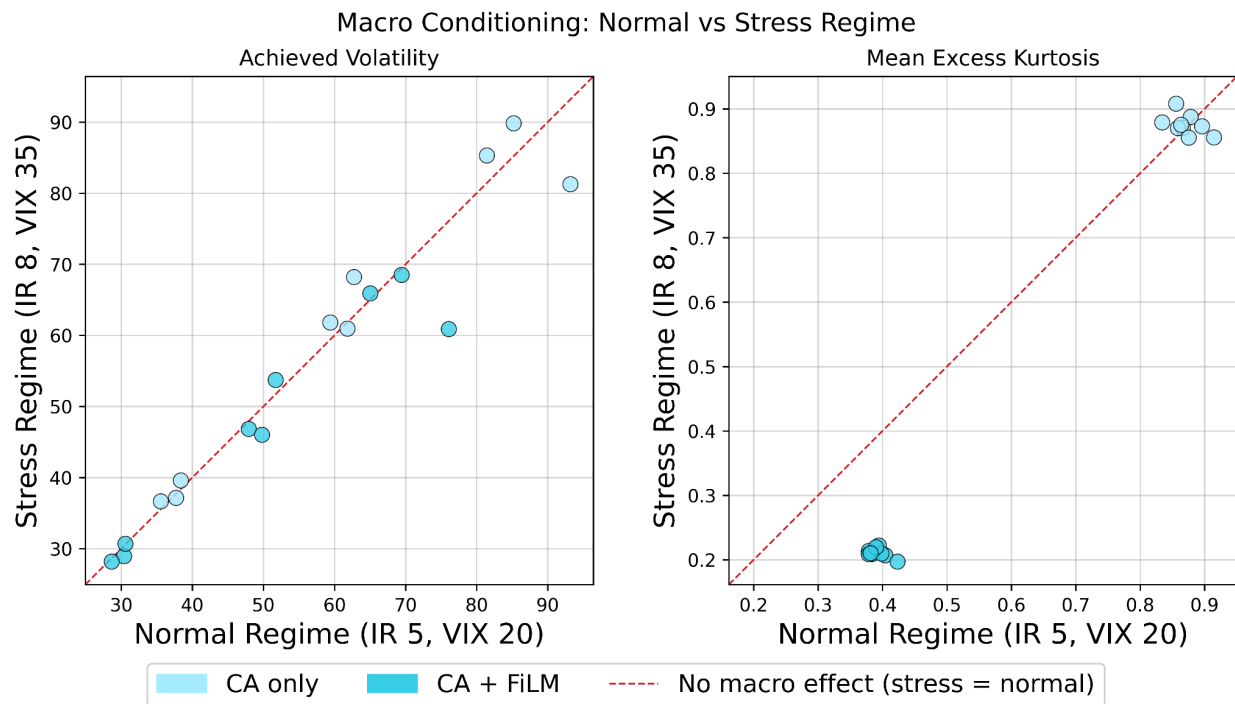
**Figure 7. Both models track target realized volatility in direction but with slight drift.**

While both models respond monotonically to the volatility condition, achieved values tend to exceed targets.

#### 4.3.2 Macro-level Condition Adherence

To evaluate responsiveness to macro-level conditioning, generated behavior is compared between specified normal and stress regimes. The normal regime is defined by a lower interest rate and lower

volatility index, while the stress regime is defined by a higher interest rate and higher volatility index. More specifically, achieved volatility and mean excess kurtosis are compared across these two regimes to measure whether generated samples respond to macro-level conditions. Figure 8 shows that macro-level conditioning remains limited for both models: normal and stress conditions do not produce a clear separation in generated volatility, and only produce a weak shift in excess kurtosis for the CA-FiLM model. These results suggest that macroeconomic conditioning is not yet strongly internalized by either model. The CA-FiLM model shows slightly more sensitivity to macro conditions than the CA-only model, supporting the idea that FiLM provides a more direct pathway for macro information. However, this shift is weak and not in the expected direction, since stress regimes are expected to produce higher excess kurtosis as a reflection of more extreme return behavior.



**Figure 8. Macro conditioning produces negligible differentiation between normal and stress regimes.**

Achieved realized volatility (left) and mean excess kurtosis (right) under normal (IR 5, VIX 20) versus stress (IR 8, VIX 35) macro conditions show clusters near the no-macro-effect diagonal across both metrics.

Upon further investigation, several possible explanations may account for this weak macro-level response, although compute constraints limit the extent to which each explanation can be tested in this project:

1. Simplified regime definitions. Intuitively, a stress regime may be associated with both high interest rates and a high volatility index, reflecting tighter financial conditions and greater market uncertainty. However, this relationship does not always hold in reality. While a high volatility index directly reflects market uncertainty, interest rates may already have been lowered during some stress periods as part of a policy response to restore market confidence. As a result, stress regimes may also appear as lower-interest-rate, high-volatility-index environments. This nuance suggests that the weak macro response may partly reflect limitations in the chosen regime specifications rather than only a failure of the model architecture.
2. Indirect relationship between macro variables and generated returns. Unlike trend and realized volatility, which are computed directly from the generated samples, interest rate and volatility index values are external market conditions that the model must learn to associate with return behavior. This relationship may be too complex for FiLM to fully capture on its own. FiLM modulates intermediate features through learned scaling and shifting parameters,  $\gamma$  and  $\beta$ , but this relatively simple linear modulation may not be strong enough to preserve complex regime information throughout the denoising network.
3. Potential conflict between micro- and macro-level conditions. Because the evaluation framework uses a grid-like conditioning space, every combination of micro- and macro-level conditions is sampled, even if some combinations rarely occur in real data. For example, specifying a low realized volatility target together with a high volatility index may create a condition pairing that is internally inconsistent. These unrealistic combinations may make macro-level effects harder to isolate, since the model must satisfy micro-level and macro-level conditions that may not naturally coexist.

## 5 Conclusion & Future Works

This project extends the CoFinDiff framework by augmenting the conditioning pathway with macro-level variables. Two model variants are proposed: one using cross-attention with both micro- and macro-level variables, and another using cross-attention for micro-level variables and Feature-wise Linear Modulation for macro-level variables. Evaluation results show that both models generate samples that closely resemble real financial data, exhibiting fat-tailed returns, volatility clustering, and a diverse sample space. The conditioning results show strong adherence to micro-level conditions, especially trend and realized volatility, but only weak responsiveness to macro-level conditions. Overall, these findings suggest that while conditional diffusion models can effectively learn direct window-level controls, macro-level conditioning remains more difficult because it requires the model to learn indirect relationships between external market variables and generated return behavior.

### 5.1 Future Work

Future work should further investigate the potential limitations identified in Section 4.3.2. Several directions are especially relevant:

- Use stronger architectures for macro-level conditioning. Future models could use deeper macro-conditioning pathways, multi-layer condition encoders, or attention-based regime embeddings to capture more complex relationships between macroeconomic variables and return behavior. This may allow macro-level information to influence the denoising process more effectively than simple FiLM modulation alone.
- Add optimization objectives beyond denoising MSE. The current models are trained primarily to predict denoising noise, which may not directly enforce condition accuracy or financial stylized facts. Future work could include auxiliary losses for achieved trend, realized volatility, kurtosis, or regime classification to encourage generated samples to better match the intended conditions.
- Improve macro-regime definitions. The normal and stress regimes used in this project are simplified representations based on interest rate and volatility index values. Future work could

define regimes using historical crisis periods, combinations of macro indicators, or clustering methods, rather than relying only on manually selected condition pairs.

- Use more realistic condition sampling. The grid-like evaluation framework samples every combination of micro- and macro-level conditions, even when some combinations may rarely occur in real markets. Future work could sample conditions from empirically observed combinations or constrain unrealistic pairings, such as low realized volatility with a high volatility index, to better reflect real market structure.

## References

- [1] S. A. Assefa, D. Dervovic, M. Mahfouz, R. E. Tillman, P. Reddy, and M. Veloso, “Generating synthetic data in finance: Opportunities, challenges and pitfalls,” in *Proc. 1st ACM Int. Conf. AI in Finance (ICAIF)*, New York, NY, USA, 2020, pp. 1–8, doi: 10.1145/3383455.3422554.
- [2] W. Chen, C. K. Yeo, C. T. Lau, and B. S. Lee, “Leveraging social media news to predict stock index movement using RNN-boost,” *Data & Knowledge Engineering*, vol. 118, pp. 14–24, 2018, doi: 10.1016/j.datak.2018.08.003.
- [3] R. Cont, “Empirical properties of asset returns: Stylized facts and statistical issues,” *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001, doi: 10.1080/713665670.
- [4] R. F. Engle and J. G. Rangel, “The spline-GARCH model for low-frequency volatility and its global macroeconomic causes,” *The Review of Financial Studies*, vol. 21, no. 3, pp. 1187–1222, 2008, doi: 10.1093/rfs/hhn004.
- [5] Y. Tanaka, R. Hashimoto, T. Takayanagi, Z. Piao, Y. Murayama, and K. Izumi, “CoFinDiff: Controllable financial diffusion model for time series generation,” in *Proc. 34th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2025, pp. 9408–9416, doi: 10.24963/ijcai.2025/1040.
- [6] R. F. Engle, “Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation,” *Econometrica*, vol. 50, no. 4, pp. 987–1007, 1982.
- [7] T. Bollerslev, “Generalized autoregressive conditional heteroskedasticity,” *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, 1986.
- [8] I. J. Goodfellow et al., “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [9] J. Yoon, D. Jarrett, and M. van der Schaar, “Time-series generative adversarial networks,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [10] M. Arjovsky and L. Bottou, “Towards principled methods for training generative adversarial networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

- [11] D. P. Kingma and M. Welling, “Auto-encoding variational Bayes,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2014.
- [12] A. Desai, C. Freeman, Z. Wang, and I. Beaver, “TimeVAE: A variational auto-encoder for multivariate time series generation,” 2021.
- [13] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.
- [14] X. Yuan and Y. Qiao, “Diffusion-TS: Interpretable diffusion for general time series generation,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
- [15] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [16] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 3942–3951.
- [18] J. B. Ramsey, D. Usikov, and G. M. Zaslavsky, “An analysis of U.S. stock price behavior using wavelets,” *Fractals*, vol. 3, no. 2, pp. 377–389, 1995.